Monolingual Transformer LMs in Hindi and Nepali

Consolidated Final Report: Phases 1–3

Language Models and Agents, Monsoon 2026: Individual Project

Model H = Hindi (higher-resource) Model L = Nepali (lower-resource)

Two independently trained, from-scratch, decoder-only Transformer LMs (∼25M params each), one per language. No pretrained weights, tokenizers, or shared vocabulary between them. Every number below is generated from metric JSON the pipeline scripts produced; I did not type any of these values by hand. Full per-phase detail: `report/phase1_report.md`, `phase2_report.md`, `phase3_report.md`.

1 Phase 1: Data Collection and Tokenizer Construction

1.1 Data Sources and Pipeline

I have chosen Hindi as the higher-resource language: IndicCorpV2 gives it three data shards plus a Wikipedia dump, so manual collection could focus on *variety* (encyclopedia and news) instead of just volume. Nepali had less to work with: IndicCorpV2 gives it only one file, and there is no Nepali Wikisource, so all manual data came from news APIs instead. This is a real constraint, not something I set up on purpose. Both languages use the same Devanagari script, and I kept that fixed so any gap I found later would be about resources, not alphabet.

I run both corpora through the same 12-step pipeline: download, normalize, filter, dedupe, split, remove train/test overlap, train the tokenizer, then tokenize. I do not allow any code under `shared/` to mention either language by name; I enforce this automatically with a test, since it is too easy to accidentally special-case one language while working fast.

I would like to explain the two choices I made here. First, *when* to remove train/test overlap: doing it before the split would let overlap-removal quietly decide which lines land in val/test, so I split first, locked val/test, and only then cleaned train against them. Second, which copy to keep when text appears twice: the manual copy always wins over IndicCorp (Hindi: Wikipedia, then news; Nepali: Onlinekhabar, then Khabarhub), a deliberate choice since the 20% manual-data target is the harder one to hit.

I set the filter thresholds by judgment, not by tuning them against results. I used a Devanagari-purity cutoff of 0.60 for Hindi but 0.70 for Nepali, lower because I expected Hindi news to mix in more English, which I later confirmed at 6.00% Latin content (§1.4). I used a fairly strict setting for duplicate detection, but a looser one for the train/test overlap check, since catching every possible overlap there matters more than being precise: a missed one means real leakage into the test set.

	Model H (Hindi)	**Model L (Nepali)**
Final train tokens (content)	632,352,544	637,535,988
Manual-lane tokens (% of train)	114,417,599 (18.1%)	116,842,022 (18.3%)
Manual sources	Wikipedia + 3 news outlets	2 news outlets (WordPress API)
Unique whitespace words	3,956,492	6,800,829
Vocabulary size (selected)	12,000	12,000
Tokenizer fertility (tok/word)	**1.3042**	**1.4394**
UNK rate (all vocab sizes)	0.0000	0.0000

Table 1: Phase 1 headline numbers. Both languages exceed the ∼500M-token and ≥20% manual-fraction targets.

1.2 Vocabulary Size Selection

I gave both models a 12,000-word vocabulary, choosing the point where bigger vocabularies stop paying off: 10K→12K still gave a solid compression gain, 14K→16K gave much less, for both languages. Past that point, every extra word costs part of the fixed ∼25M-parameter budget. I also chose one shared size for both models so a later perplexity gap could not be blamed on one model simply having a bigger vocabulary.

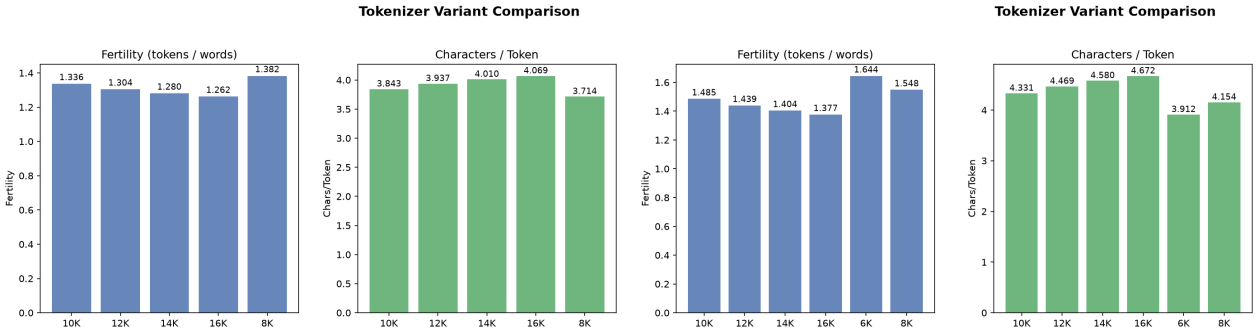


Figure 1: Fertility and chars/token across candidate vocabulary sizes, held-out validation. Hindi (left) is consistently more compressible than Nepali (right) at every matched vocabulary size.

1.3 Tokenizer Fertility and Morphology

One pattern I keep seeing throughout this project: Nepali needs more tokens per word than Hindi (10.4% more, on average). I traced this to grammar, not bad data. Nepali sticks small grammatical pieces onto the end of a word with no space (so "Nepal" plus "of" becomes one word, नेपालको), while Hindi keeps that piece separate. This is also why Nepali has more unique words than Hindi (6.80M vs. 3.96M): sticking pieces together creates more distinct word forms for the tokenizer to cover. Example: Hindi's भारत एक विशाल देश है। splits into 6 whole-word tokens; the similarly short Nepali नेपाल एक सुन्दर हिमाली देश हो। takes 7. I found both tokenizers struggle equally on modern AI/ML terms absent from either corpus, e.g. "machine learning" breaking into 6–10 fragments.

1.4 Corpus Health Check

Before trusting the above, I checked the corpus itself. Devanagari purity is 90% or higher in every source, so the script filter works. Hindi's news carries 6.00% Latin-script content, mostly English names mixed into sentences; Nepali's news is cleaner (≥98.8% Devanagari), likely a style difference rather than a quality one. The bigger finding is document length: news articles average 1,499–2,494 characters against IndicCorp's 218–286-character fragments, so one manual article is worth roughly 7–10 IndicCorp entries. That is why manual data reaches ∼18% of tokens while being only 3–8% of documents collected, and why every "manual fraction" number here is counted **by token**, not by document.

1.5 Requirement Compliance

Requirement	Hindi	Nepali	Status
∼500M training tokens	632,352,544	637,535,988	**PASS** (exceed by 26–28%)
IndicCorp sub-target 400M	517,934,945	520,693,966	**PASS**
Manual sub-target ≥100M	114,417,599 (+14.4%)	116,842,022 (+16.8%)	**PASS**
Tokenizer trained from scratch	yes	yes	**PASS**
No shared tokenizer/vocab/data	yes	yes	**PASS**
Train decontaminated vs. val/test	yes	yes	**PASS**

Table 2: Phase 1 requirement compliance, both languages.

2 Phase 2: Model, Pretraining, and Evaluation

2.1 Architecture

I built the model as a decoder-only Transformer from basic building blocks, with no ready-made Transformer layer and no library like HuggingFace, since the point of this phase was for me to understand attention and masking directly rather than hide them behind a library.

It has 6 layers and 8 attention heads, about **25.3 million** parameters, identical for both languages so any later difference comes from data and language, not architecture. I checked causal masking (each token sees only earlier tokens) directly: changing a future token and re-running confirms earlier predictions do not change.

I chose a wider, shallower model (6 layers) over a narrower, deeper one (12 layers) for two reasons: tying the input and output word-embeddings together frees up parameters better spent on width, and with ∼650M training tokens for a 25M-parameter model, there is already more than enough data, so extra depth would not help much. I also used a training-stability trick called pre-norm, since a crashed run on a cloud GPU means starting over from scratch, and a simple learned lookup table for positions rather than a fancier method, for easier debugging, at the cost of a 512-token limit this project never actually reaches.

2.2 Pretraining Setup

For pretraining, I used the AdamW optimizer, a warmup period, slow learning-rate decay, and mixed-precision training. I ran training for exactly one epoch, since both corpora already exceed what a model this size needs, so a second pass would mostly encourage memorization. I tested the checkpoint-and-resume logic to confirm it produces identical results whether training runs straight through or is stopped and resumed, since cloud sessions can be cut off without warning.

Metric	**Model H (Hindi)**	**Model L (Nepali)**
Training tokens (1 epoch)	648,012,574	659,499,440
Optimizer steps	13,183	13,417
Training wall-clock (h, 2×T4)	2.56	2.62
Validation cross-entropy (nats/token)	3.2662	3.7262
Validation perplexity	26.21	41.52
Validation bits-per-byte (BPB)	0.4797	0.4635
Test perplexity / BPB	26.19 / 0.4791	41.46 / 0.4632
Mean attention entropy (nats)	1.3109	1.2671
Mean attention distance (tokens)	3.98	3.84

Table 3: Phase 2 headline results, validation and test.

2.3 Language-Modeling Results

The most interesting thing I found in this phase: which model looks ”better” flips depending on how I measure it. On perplexity, Nepali looks 58.4% worse (41.52 vs. 26.21). But perplexity is measured in each model's own tokenizer, and Nepali's tokenizer needs 10.4% more tokens for the same text, so it is working with less real content per step, unrelated to how well it actually learned. When I switch to bits-per-byte, a measure based on raw text bytes instead of tokens, the result flips: Nepali is actually 3.4% *better*. This holds on the test set too, and steadily from partway through training onward, not just at the end. I see the same idea show up again, differently, in the Phase 3 results (§3).

2.4 Generation Quality and Attention

To test generation, I had both models complete 500 held-out prompts at settings from safe (greedy) to more random (higher temperature), and I scored the output against the real continuation using each

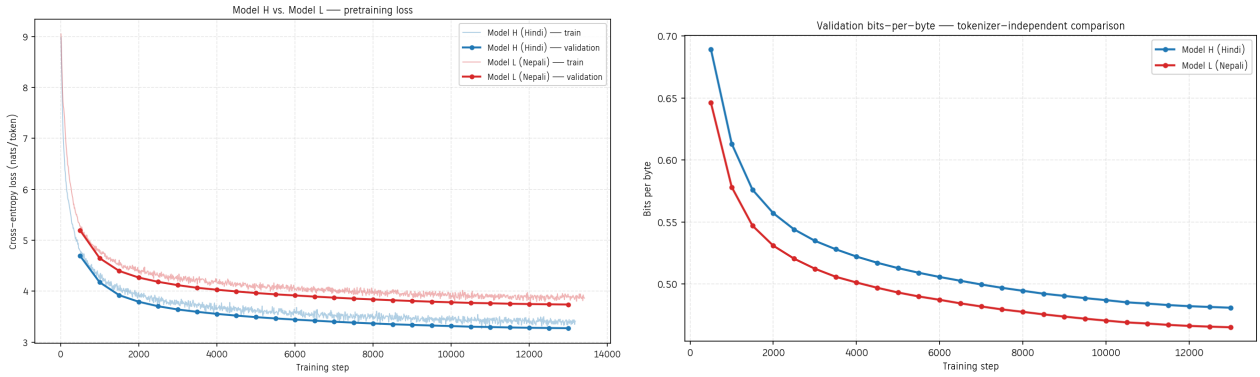


Figure 2: Left: training loss, both models, smooth convergence. Right: validation BPB over training; the Model L < Model H ordering holds from step 500 onward.

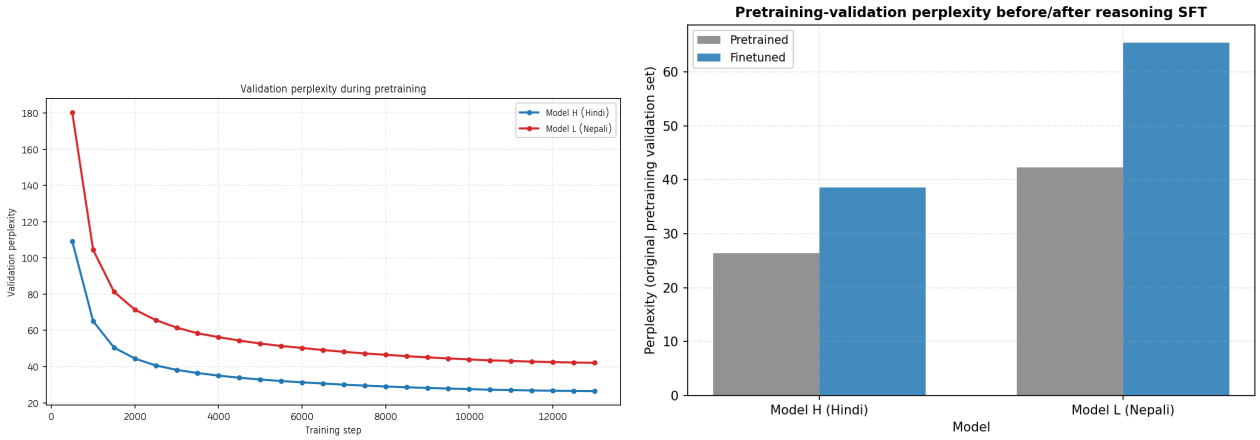


Figure 3: Left: validation perplexity over training, token space, both models (Model H ahead throughout, the mirror image of the BPB plot above). Right: the same pretraining-validation perplexity re-measured after Phase 3 SFT, used later as the catastrophic-forgetting diagnostic in §3.

language's own subword pieces rather than whitespace words, since whitespace scoring would unfairly penalize Nepali's larger word inventory. I found BLEU close to useless here: a small model rarely matches a reference continuation word-for-word, so scores stayed low across the board, an honest limit of the metric, not a sign the models are bad. chrF, which gives partial credit for a correct stem with a different ending, was far more useful to me, since that near-miss is common in both languages' grammar, and was the only metric that reliably improved with more random sampling.

I found repetition mattered more than any overlap score. Under greedy decoding, both models tend to get stuck repeating the same phrase (roughly 70–76% of the time), something overlap metrics barely notice but a diversity metric catches clearly, dropping sharply once sampling gets more random. One real Hindi example I looked at: given a prompt about the rural economy, the model produced a grammatical, on-topic sentence but visibly repeated "for development" instead of finishing the thought. On attention, I found both models' heads split into nearby-focused and farther-focused groups; Nepali leans more toward nearby words overall, fitting the earlier finding that a Nepali word split into more pieces needs its next-piece clue close by.

3 Phase 3: Reasoning Finetuning, Attention Analysis

3.1 Reasoning Dataset Construction

Instead of using an existing benchmark, I built my own reasoning dataset per language, so I could control the correct answer. The task is simple comparisons: given two or three people or things and a

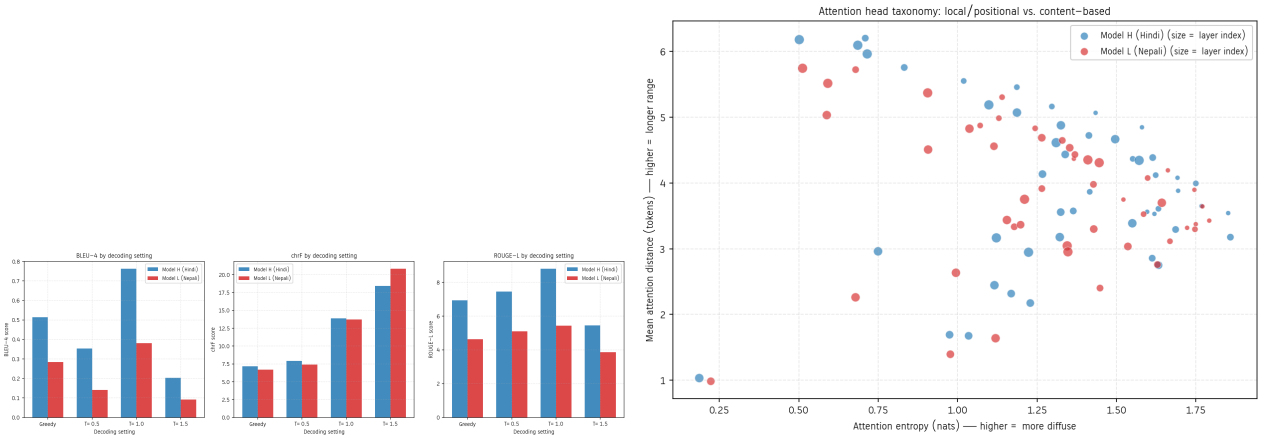


Figure 4: Left: generation-quality metrics (BLEU-4, chrF, ROUGE-L) across decoding settings, both models. Right: local/positional vs. content-based head-taxonomy split, both models, 6 layers × 8 heads each.

property (age, height, price, quantity), say which is biggest or smallest. I wrote four possible phrasings for each question; I use three for training and hold one out entirely, to test unseen wording. This gives me 2,400 training examples and 320 test examples in each of three categories: seen names and wording, unseen names, and unseen wording, with no overlap between the training and test names I used. Keeping these separate lets the three accuracy numbers in Table 5 fail independently instead of blending into one number.

3.2 Finetuning Setup

I finetune each model from its own pretrained checkpoint, keeping the tokenizer and vocabulary unchanged. I only grade the answer part of each example during training, though the model still reads the full question. I used full finetuning (every weight updated) rather than a lighter memory-saving method, which I judged unnecessary at this model's small size. I gave both languages identical settings, so any later difference comes from data and language, not training choices.

Hyperparameter	**Model H (Hindi)**	**Model L (Nepali)**
Learning rate (peak → min)	$10^{-4} \rightarrow 10^{-5}$	$10^{-4} \rightarrow 10^{-5}$
Weight decay	0.01	0.01
Warmup ratio	5%	5%
Epochs	4	4
Batch size	16	16
Gradient clip (max norm)	1.0	1.0
Max sequence length	128	128
Seed	42	42
Optimizer	AdamW (0.9, 0.95)	AdamW (0.9, 0.95)
LR schedule	cosine decay	cosine decay
Total training steps	600	600
Resumed from pretrained step	13,000	13,000
Best validation loss	0.3722	0.3740
Wall-clock time (CPU)	721.4 s (∼12 min)	1491.2 s (∼25 min)

Table 4: Finetuning hyperparameters, both languages, from the corrected/retrained run. Nepali takes roughly twice as long wall-clock for the same 600 steps, consistent with higher tokenizer fertility.

3.3 Fixing a Memorization Failure

This setup is the result of me fixing a real mistake. An earlier version I ran, with only 24 names and 6 epochs, scored well on seen names but exactly **0.000** on both unseen tests, a classic memorization sign: with names repeated so often, the model learned to guess a familiar one instead of comparing. I fixed

it with many more names (80 instead of 24) and fewer epochs at a lower learning rate. Every number below is from this corrected run.

Test split	Model	Pretrained	Finetuned	Δ
Seen entity + seen wording	H (Hindi)	0.000	0.412	+0.412
Unseen entity + seen wording	H (Hindi)	0.000	0.066	+0.066
Unseen entity + unseen wording	H (Hindi)	0.000	0.059	+0.059
Seen entity + seen wording	L (Nepali)	0.000	0.434	+0.434
Unseen entity + seen wording	L (Nepali)	0.000	0.191	+0.191
Unseen entity + unseen wording	L (Nepali)	0.000	0.203	+0.203

Table 5: Three-way reasoning exact-match accuracy, pretrained-only vs. finetuned (mean over 3 seeds; greedy decoding, so seeds are a correctness check, not a variance estimate).

3.4 Reasoning Accuracy Results

The most surprising result I found in the whole project: Model L (Nepali) handles both unseen tests *better* than Model H, despite trailing Hindi on every raw data metric in Phases 1–2. I re-ran the evaluation and got the exact same numbers, so I am confident this is real, not noise. My best explanation is that this reflects what each model already understood before finetuning started, not that Nepali's language modeling is stronger; I already showed the opposite on perplexity in Phase 2, and the two were close to tied on the fairer byte-based measure.

	Before SFT	After SFT
Model H pretraining-val PPL	26.38	38.55
Model L pretraining-val PPL	42.32	65.47

Table 6: Catastrophic-forgetting diagnostic: PPL on the *original* Phase 2 pretraining validation set, before/after reasoning SFT. Expected to rise; this measures distribution shift, not reasoning success.

3.5 Catastrophic-Forgetting Check

I found these reasoning gains are not free: both models get noticeably worse at general language modeling on the original Phase 2 validation text, which I expected after a few epochs on a small, repetitive task. Hindi and Nepali worsen by a similar amount, so this tells me little about which language "forgets better," but it confirms the new reasoning skill cost real fluency.

3.6 Error Analysis

Looking at specific wrong answers made this gap easier for me to see. In one Hindi example, the model answers with a common training-data name instead of either person actually named in the question, as if it learned "this name is usually the answer" rather than comparing. Nepali sometimes makes the same mistake, but other times gives a small typo-like distortion of the correct name instead, suggesting it was trying to recall the answer and got it slightly wrong, rather than ignoring the question. Table 7 gives a broader sample.

To check this properly rather than eyeballing a few examples, I wrote a script that sorts *every* wrong answer into one of four categories: a random training-data name, a garbled attempt at the right name, the *other* real name in the question, or something else.

The results confirmed the pattern I suspected: 77–85% of Hindi's wrong answers are a familiar training-data name, almost never a garbled attempt at the right one. Nepali is different: it names the *other* real thing from the question far more often (25–45% vs. 5–9% for Hindi), which tells me it is at least reading the question, just picking the wrong side, matching its stronger overall performance.

Lang.	Prompt (unseen-entity split)	Gold	Predicted
H	अगर शांति स्नेहा से छोटी है, तो इनमें से बड़ा कौन है?	स्नेहा	स्नेहा ✓
H	अगर गणेश विमला से लंबा है, तो इनमें से लंबा कौन है?	गणेश	गणेश ✓
H	अगर अभिषेक शीला से बड़ा है, तो इनमें से बड़ा कौन है?	अभिषेक	अमित ×
H	अगर किरण ललिता से बड़ी है, तो इनमें से बड़ा कौन है?	किरण	तरिता ×
H	अगर प्रिया स्नेहा से छोटी है, तो इनमें से बड़ा कौन है?	स्नेहा	नेहा ×
H	अगर सुमित शीला से छोटा है, तो इनमें से छोटा कौन है?	सुमित	अमित ×
L	यदि राधा सुमितभन्दा कान्छी छ भने, यीमध्ये जेठो को हो?	सुमित	सुमित ✓
L	यदि कम्बल जुत्ताभन्दा थोरै छ भने, यीमध्ये कसको मात्रा थोरै छ?	कम्बल	कम्बल ✓
L	यदि उर्मिला रेखाभन्दा जेठी छ भने, यीमध्ये जेठो को हो?	उर्मिला	रीता ×
L	यदि आरती यशभन्दा जेठी छ भने, यीमध्ये कान्छो को हो?	यश	जश ×
L	यदि मंजू प्रियाभन्दा कान्छी छ भने, यीमध्ये कान्छो को हो?	मंजू	संजय ×
L	यदि गायत्री आरतीभन्दा कान्छी छ भने, यीमध्ये जेठो को हो?	आरती	लस ×

Table 7: Unseen-entity predictions, both languages, finetuned checkpoints. Hindi's wrong answers are all real names from the training pool; Nepali's mix training-pool names with malformed near-misses and outright non-answers.

Lang.	Task	n wrong	Train-pool halluc.	Malformed	Legal wrong entity	Empty/other
H	pairwise	291	77.3%	9.3%	9.3%	4.1%
H	transitive3	309	85.4%	3.6%	4.9%	6.1%
L	pairwise	240	58.8%	8.3%	25.4%	7.5%
L	transitive3	274	47.4%	2.2%	44.9%	5.5%

Table 8: Error-provenance classification. Hindi's errors are dominated by hallucinating a training-pool name (memorization); Nepali more often names the *other* real entity in the prompt (25–45%) rather than hallucinating, a more prompt-engaged failure mode consistent with Nepali's stronger unseen-split accuracy.

3.7 Attention Analysis After Finetuning

To see how finetuning changes the model internally, I ran the same attention-analysis tool on the pretrained and finetuned versions, using the same comparison-question both times, so any difference I find is due to finetuning alone. Before finetuning, deeper layers spread attention loosely across the sentence, often anchored on the first word, a pattern I also saw in Phase 2. After finetuning, in both languages without being told to, attention becomes more focused on specific, relevant words (a "spread" score drops from about 1.59 to 1.56 for Hindi, roughly flat for Nepali) and shifts slightly further back, from about 4.9 tokens to 5.0–5.1. I read this as a plausible story: the model learns to actively search for and lock onto the two things being compared.

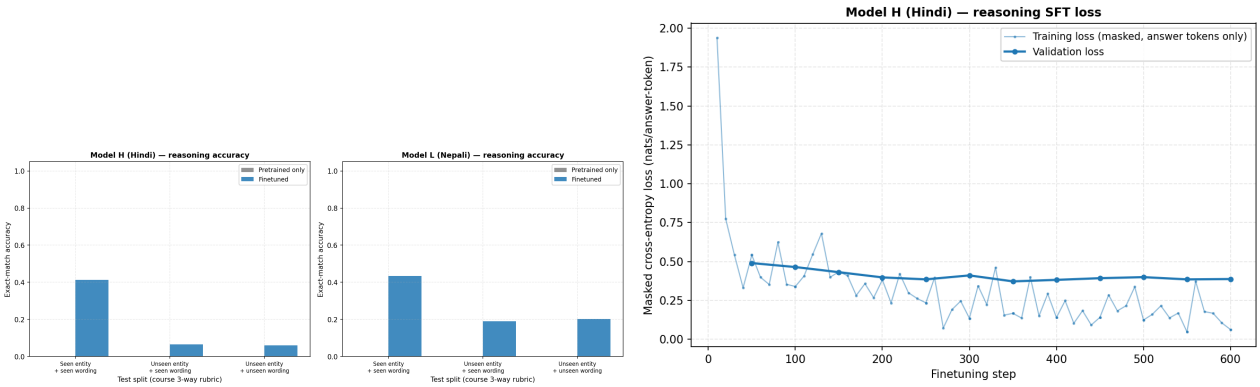


Figure 5: Left: three-way accuracy comparison, both models. Right: Hindi SFT loss curve; the Nepali curve (not shown) follows the same smooth-convergence shape over the same 600 steps.

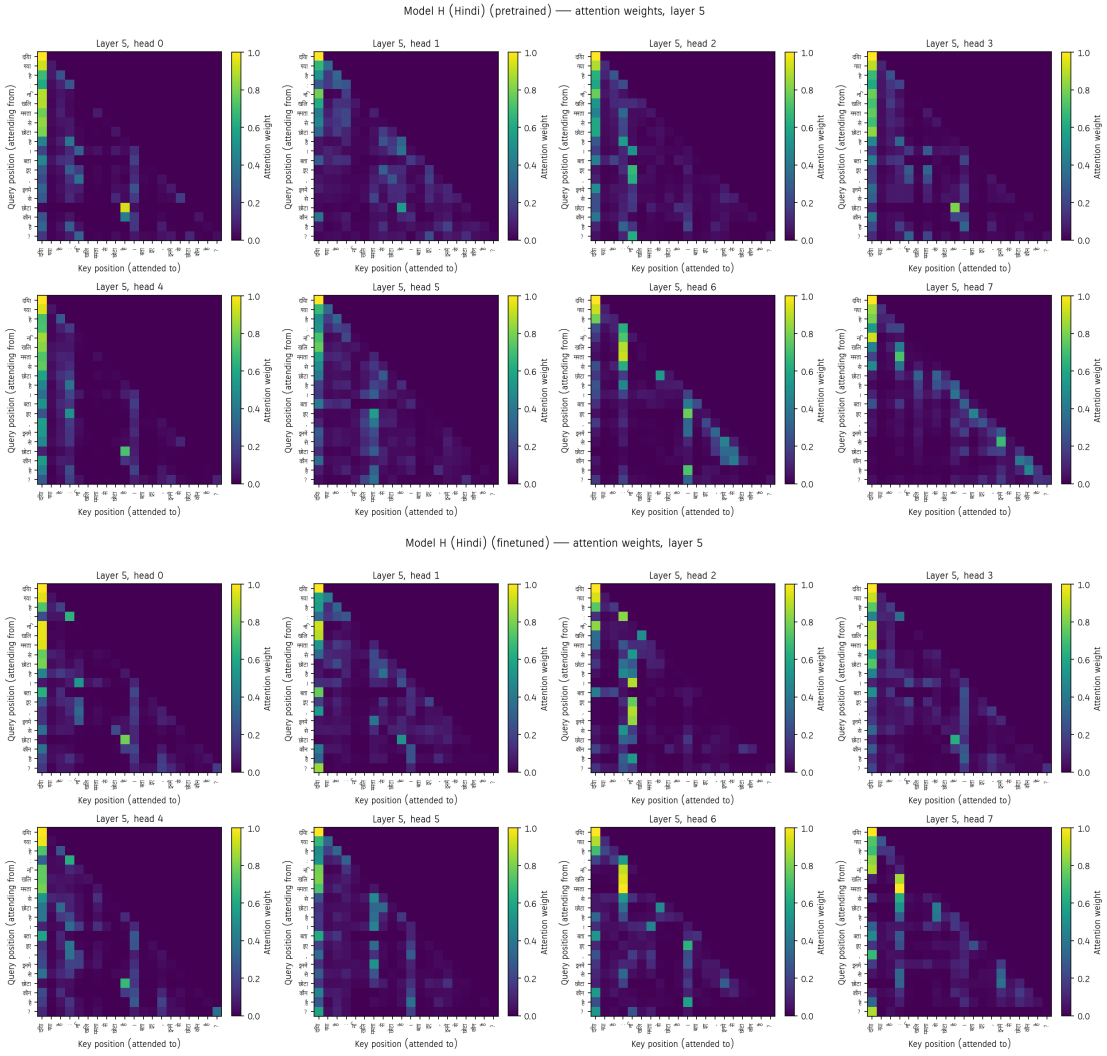


Figure 6: Hindi, layer 5, same probe sentence: attention before finetuning (top, diffuse first-token sink) vs. after (bottom, sharper content-specific anchor), the sharpen-and-lengthen pattern described above.

It is worth noting that this shift happens on top of an already uneven starting point, not a flat one. Even before finetuning, I found how far each head looks back varies a lot by layer, dipping toward nearby words in the middle layers before widening again at the last layer, in both languages.

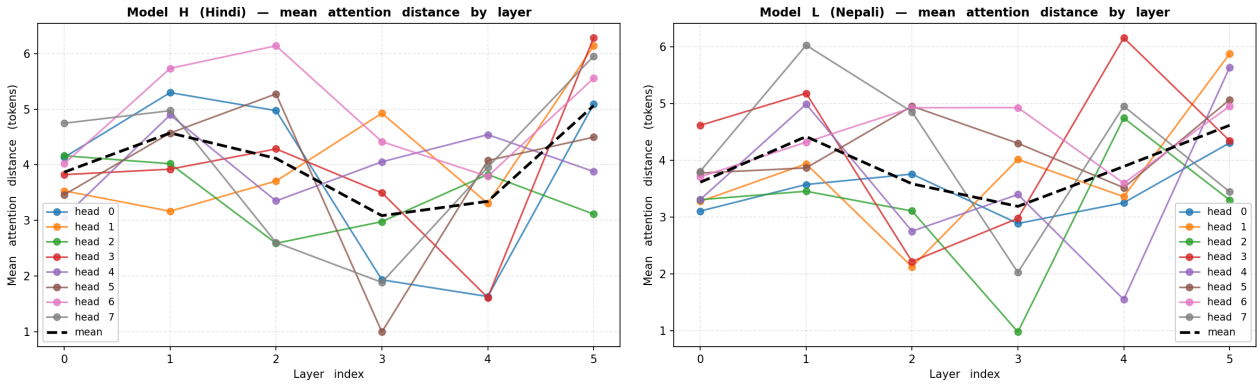


Figure 7: Per-head mean attention distance by layer, pretrained checkpoint, Hindi (left) and Nepali (right).

4 Cross-Phase Metrics Comparison

I put every Hindi-vs-Nepali number from all three phases in one table below. It makes one thing clear to me: there is no simple story where the "higher-resource" model just wins everywhere; it depends on the metric.

Phase	Metric	Model H	Model L	Favors / gap
1	Train tokens	632.4M	637.5M	tied (<1% apart)
1	Manual-lane share	18.1%	18.3%	tied
1	Unique whitespace words	3.96M	6.80M	L has 1.7× more (morphology)
1	Tokenizer fertility (tok/word)	**1.3042**	1.4394	**H** by 10.4% (fewer tokens/word)
2	Validation perplexity	**26.21**	41.52	**H** by 58.4% (token space)
2	Validation BPB (byte space)	0.4797	**0.4635**	**L** by 3.4% (tokenizer-fair)
2	Mean attention entropy	1.3109	1.2671	L sharper (more local)
2	Generation chrF range (greedy→T1.5)	7.16–18.44	6.71–20.84	L wider spread at high T
2	Greedy repetition rate	**0.70**	0.76	**H** slightly less repetitive
3	Reasoning acc., seen/seen	0.412	**0.434**	**L** by 0.022
3	Reasoning acc., unseen entity	0.066	**0.191**	**L** by 0.125 (2.9×)
3	Reasoning acc., unseen wording	0.059	**0.203**	**L** by 0.144 (3.4×)
3	Forgetting: PPL rise factor after SFT	1.46×	1.55×	H forgets slightly less
3	Error type: train-pool hallucination	77–85%	47–59%	H memorizes more

Table 9: All H-vs-L metrics across the project, one table. **Bold** marks the favored value per row. Every Phase 1 raw-scale metric is tied, every Phase 2 *token-space* metric favors H, but BPB and all three Phase 3 reasoning-generalization metrics favor L: the resource-tier story is not monotonic across metrics.

5 Final Comparison: the Four Required Questions

1. How did data scale and quality differ between Model H and Model L? I found that the amount of data was not the real difference: Hindi had 648.0M training tokens, Nepali 659.5M, so if anything Nepali had slightly more. The real difference is grammar: Nepali sticks endings onto words, creating far more unique word forms (6.80M vs. 3.96M) from a similar-sized corpus, which is also why its tokenizer needs 10.4% more tokens per word.

2. How do language-modeling and reasoning results compare across the two resource tiers? On perplexity, Nepali looks 58.4% worse to me; on the fairer byte-based measure, it is actually 3.4% *better*. On reasoning, the first test that measures real learned skill rather than pattern-fitting, Nepali also comes out ahead on both unseen categories. Across every metric I collected (Table 9), only token-based perplexity clearly favors Hindi; scale metrics are tied, and the byte-based measure plus all reasoning results favor Nepali.

3. What tokenizer/corpus factors most affected the lower-resource model? I found Nepali's higher token-per-word count explains the perplexity-vs-byte flip above, but it does not explain a separate gap I found on the harder three-way comparison task. I tested one guess, that a specific Nepali grammatical word tokenizes inconsistently, and ruled it out. I am reporting that gap honestly as open rather than forcing it into an explanation that does not hold up.

4. What evidence explains the observed differences? The most convincing evidence for me came from reading individual wrong answers. Hindi mostly repeats a familiar training-data name (77–85% of mistakes), which I read as pointing to memorization. Nepali leans more toward naming the *other* real thing in the question (25–45%), still engaging with the question even when wrong. I also found a second, independent signal that agrees: after finetuning, both models' attention shifts to focus more precisely on relevant words, direct evidence the internal processing changed, not just the outputs.

6 Where Automation Substitutes for Human Judgment

Here are the places where I let automatic rules stand in for actually reading the text myself, a real limitation I want to state directly:

- I did not read a sample of kept or rejected text by hand (Phase 1); my filter settings are standard choices, not values I tuned by inspection.
- I judged generation quality (Phase 2) entirely by metrics; the example sentences I show prove the output is on-topic, not a substitute for native-speaker judgment.
- My attention-head labels (Phases 2–3) come from a simple statistical split, not a head-by-head check of whether each label is meaningful.
- I had a fluent speaker review the reasoning questions (Phase 3) for naturalness, but only the 12 underlying patterns, not all ∼6,000 examples; this caught one real Nepali grammar bug, which I fixed before producing the results above.
- I tested one guess for the extra accuracy gap on the harder three-way task and ruled it out; I found no confirmed explanation, so I am reporting it as open rather than papering over it.